



Kubernetes API ve Manifest Okuryazarlığı

Kubernetes ve Konteyner Dünyası



Bu Konuda

- Kubernetes nesnelerinin yapısı
- API resource, object, group ve version kavramları
- Imperative ve declarative yönetim
- kubectl ile keşif ve manifest üretme
- Server-Side Apply
- Namespace, label, annotation ve selector

Kubernetes Nesnelerinin Yapısı





Kubernetes API'si ve Nesneler

- API server, Kubernetes control plane'in **çekirdeği**dir; bir HTTP API sunar^[1]
- `kubectl` ve istemci kütüphaneleri bu API üzerinden nesneleri sorgular ve değiştirir^{[1],[12]}
- Kubernetes **nesneleri**, cluster durumunu temsil eden kalıcı varlıklardır; bir "record of intent" (niyet kaydı)^[2]
- Nesne oluşturmak **desired state**'i bildirir; sistem sürekli olarak gerçekleşmesi için çalışır^[2]
- API server, nesnelerin durumunu **etcd**'de saklar^[1]



Nesne Yapısı

Her manifestte gereken alanlar: ^[2]

- **apiVersion** - nesnenin oluşturulacağı API sürümü
- **kind** - oluşturulacak nesne türü
- **metadata** - nesneyi tanımlayan veri: **name**, **UID**, isteğe bağlı **namespace**
- **spec** - istenen durum; alan yapısı nesne türüne göre değişir

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - name: nginx
    image: nginx
```



spec ve status Ayrımı

- **spec** oluştururken belirlenir; nesnenin **istenen durum**udur (desired state) ^[2]
- **status** sistemin sürekli güncellediği **mevcut durum**dur (actual state) ^[2]
- Control plane, her nesnenin gerçek durumunu istenen duruma yaklaştırmak için çalışır ^[2]
- Örnek: Deployment 3 replica ister; sistem 3 başlatıp **status** 'ü günceller; bir örnek ölürse düzeltme yapar ^[2]



API Resource ve API Object

- **API resource**, API'deki bir nesne türüdür (ör. `Pods`)^[1]
- **API object**, o türün bir örneğidir (ör. adı `nginx` olan bir Pod)^[1]
- Discovery API, her resource için isim, scope, desteklenen fiiller ve group/version/kind yayınlar^[1]
- kubectl, desteklenen kaynak listesini bu discovery verisinden alır^[1]



API Group ve Version

- Sürümleme **API seviyesinde** yapılır; her nesne alanı için değil^[1]
- Çekirdek (core) API: `/api/v1`; gruplu API'ler: `/apis/<group>/<version>` ^[1]
- Aynı veri birden çok sürümle gösterilebilir; API server dönüşümü yapar^[1]
- `v1beta1` ile oluşturulan nesne, beta deprecate edilene dek `v1` ile de okunabilir^[1]
- **GA (v1)**: uyumluluk garantisi verir. **Beta**: verisi GA'ya dönüştürülebilir. **Alpha**: uyumluluk garantisi yok^[1]



YAML Temelleri

Manifestler YAML'dır (JSON da geçerli); kubectl, HTTP isteğine çevirir. ^[2]

```
app: nginx                # sözlük (map): anahtar: değer
replicas: 3               # sayı
tags:                     # liste (array): "-" ile
- frontend                # liste öğeleri anahtarla aynı girintide olabilir
- backend
spec:
  containers:             # iç içe sözlük + liste
  - name: nginx
    image: nginx:1.27
```

- Girintileme önemlidir; boşluk kullanılır, tab kullanılmaz
- **Tek dosyada birden çok kaynak** `---` ayracıyla tanımlanır
- Dosyalar `-f <dosya>` ile komutlara verilir (ör. `kubectl apply -f`, `kubectl create -f`)



Nesne İsimlendirme

- **DNS subdomain (RFC 1123)**: en fazla 253 karakter; küçük harf, rakam, `-`, `.`; alfanümerikle başlar ve biter - ör. `my-app.example.com` geçerli; `MyApp`, `-app` geçersiz^[4]
- **DNS label (RFC 1035/1123)**: en fazla 63 karakter; küçük harf, rakam, `-`; **harfle başlar** - ör. `web-1` geçerli; `1web`, `web_1` geçersiz^[4]
- **generateName**: `name` yerine ön ek verilir; sunucu benzersiz bir son ekler; çakışmada HTTP 409 döner^[4]
- **UID**: kullanıcı belirlemez, **sistem atar**; nesneyi cluster genelinde benzersiz tanımlar (UUID'dir)^[4]
- **Nesne kimliği** (kullanıcının referansı): API group + resource türü + namespace + isim; **UID** bu dörtlünden ayrı, sistemin atadığı benzersiz kimliktir^[4]

Yönetim Yaklaşımları





Imperative ve Declarative Yönetim

Bir Kubernetes nesnesi **tek teknikle** yönetilmeli; teknikleri karıştırmak tanımsız davranışa yol açar. ^[8]

Yaklaşım	Çalıştığı yer	Önerilen ortam	Yazarlar	Öğrenme
Imperative komutlar	Canlı nesneler	Geliştirme	1+	En düşük
Imperative nesne yapılandırması	Tek dosyalar	Gerçek ortam	1	Orta
Declarative nesne yapılandırması	Dosya dizinleri	Gerçek ortam	1+	En yüksek

- **Imperative:** `kubectl create / run` - hızlı, tek adım; ancak geçmiş ve denetim (audit) izi yok ^[8]
- **Declarative:** `kubectl apply` - Git'te tutulur, denetlenebilir, GitOps ile uyumlu ^{[8],[12]}
 - **Ekosistem ve Servis Sağlayıcıları**



API Keşfi

- `kubectl explain` - bir nesnenin alan dokümantasyonunu gösterir^[13]
- `kubectl api-resources` - cluster'ın desteklediği kaynakları listeler^[13]
- `kubectl api-versions` - sunucudaki API sürümlerini listeler

```
kubectl explain pods  
kubectl explain pod.spec.containers  
kubectl api-resources  
kubectl api-versions
```



Manifest Üretme

- `--dry-run=client -o yaml`, isteği sunucuya göndermeden manifest üretir^[13]

```
kubectl create deployment nginx --image=nginx --dry-run=client -o yaml > nginx.yaml  
kubectl run nginx --image=nginx --dry-run=client -o yaml > pod.yaml
```

- Üretilen dosya kaydedilip `kubectl apply` ile declarative yönetilebilir
- `--dry-run=server` isteği sunucuya iletir ve doğrulama/admission kontrollerini çalıştırır
→ **Pod Güvenliği ve Observability**



kubectl apply ve diff

- `kubectl apply`, dosyadaki istenen durumu uygular; nesne yoksa oluşturur, varsa günceller^[13]
 - apply **declarative**dir: operasyon belirtilmez; oluşturma ve güncelleme duruma göre algılanır^[8]
- `kubectl diff`, uygulamadan önce değişikliği gösterir^[8]

```
kubectl diff -f ./nginx.yaml
kubectl apply -f ./nginx.yaml
kubectl apply -f ./configs/
```

- apply, `patch` API kullanır: yalnızca gözlenen farklar uygulanır; başka yazarların değişiklikleri korunur^[8]



replace, edit ve patch

- `kubectl replace` - dosyadaki tam nesneyle değiştirir; dosyada olmayan canlı değişiklikler **kaybolur**^{[8],[13]}
- `kubectl edit` - canlı nesneyi metin düzenleyicide açar^[13]
- `kubectl patch` - kısmi güncelleme yapar (JSON merge / strategic merge)^[13]

```
kubectl replace -f ./nginx.yaml
kubectl edit deployment nginx
kubectl patch node k8s-node-1 -p '{"spec":{"unschedulable":true}}'
```

- Uyarı: bağımsız güncellenen alanlarda (ör. LoadBalancer `externalIPs`) `replace` kullanılmamalı^[8]



Declarative Kaynaklarda Manuel Değişikliklerin Etkisi

- Bir nesne için **tek teknik** kuralı ihlal edilirse (ör. apply ile yönetilene `replace` uygulamak) davranış tanımsızlaşır^[8]
- `replace` ile yapılan canlı değişiklikler dosyaya yansıtılmazsa sonraki replace'te kaybolur^[8]
- `apply` ise yalnızca gözlenen farkı uygular; dosyada olmayan başka değişiklikleri korur^[8]



Server-Side Apply: Field Ownership

- SSA, birden çok uygulayıcının tek nesneyi **birlikte yönetmesini** sağlar: her uygulayıcı **kendi uyguladığı alanların** sahibi olur - aynı nesnenin farklı alanları farklı yöneticilerde olabilir^[14]
- `metadata.managedFields`, alan yönetimi bilgisini taşır: `manager`, `operation`, `fieldsV1` ^[14]

```
metadata:
  managedFields:
  - manager: kubectl
    operation: Apply
    fieldsV1:
      f:spec:
        f:replicas: {}
  - manager: kube-controller-manager
    operation: Update
    fieldsV1:
      f:status:
        f:readyReplicas: {}
```

- Bir alanın değerini değiştiren **son yönetici** onun sahibi olur^[14]
- `kubectl get --show-managed-fields` ile görüntülenir^[14]



SSA: Paylaşımlı Sahiplik

- Bir alana **aynı değeri** yazan birden çok uygulayıcı, çakışma olmadan **paylaşımlı sahipliğe** geçer; alan her birinin `managedFields` kaydında görünür^[14]

```
metadata:
  managedFields:
  - manager: kubectl
    operation: Apply
    fieldsV1:
      f:spec:
        f:replicas: {}
  - manager: helm
    operation: Apply
    fieldsV1:
      f:spec:
        f:replicas: {}
```

- Paylaşım, diğer alanları uygulamayı **engellemez**; çakışma yalnızca paylaşılan alanı **farklı bir değerle** değiştirmeye çalışınca çıkar^[14]



SSA Çakışmaları ve `--force-conflicts`

- Apply, başka bir yöneticinin sahip olduğu bir alanı **farklı bir değerle** değiştirmeye çalışırsa **çakışma (conflict)** hatası döner; değişiklik uygulanmaz^[14]
- Çakışma yalnızca **başka bir yöneticinin sahipliğindeki** alanlar için çıkar; uygulayıcının kendi alanlarını değiştirmesi normaldir (ör. kendi uyguladığı Deployment'ın `spec.replicas` 'ı)
- Üç çözüm yolu vardır^[14]:
 - i. **Zorla:** `--force-conflicts` ekle; alan değişir, sahiplik uygulayana geçer
 - ii. **Birak:** çakışan alanı manifest'ten çıkarıp yeniden uygula; o alan üzerinde iddia bırakılır (**paylaşımlı sahiplikte kayıttan da düşer**) - değer başka yönetici sahip olduğu için korunur
 - iii. **Paylaş:** manifest'teki değeri sunucudakiyle aynı yapıp yeniden uygula; alan paylaşımlı sahipliğe geçer

```
kubectl apply --server-side -f nginx.yaml  
kubectl apply --server-side --force-conflicts -f nginx.yaml
```

- `--server-side` SSA'yı açar; `--force-conflicts` yalnızca 1. yoldur (2. ve 3. bayraksız manifest düzenlemesidir)
- Güncelleme (PUT) çakışmada başarısız olmaz; **apply** çakışmada başarısız olur^[14]



Doğrulama: Client-side ve Server-side

- v1.25+ API server, **tanınmayan veya tekrarlanan alanları** algılayan sunucu tarafı doğrulama sunar^[2]
- `kubectl --validate: ignore, warn, strict`; varsayılan `true` (= `strict`)^[2]
- `strict` hatada başarısız olur; `warn` uyarı verir; `ignore` atlar^[2]
- Sunucu doğrulamayı desteklemiyorsa kubectl istemci tarafına düşer; Kubernetes 1.27+ her zaman sunar^[2]

Cluster Üzerindeki Yapı Taşları





Namespace ve Kaynak Kapsamı

- Namespace, kaynakları **tek cluster içinde gruplara** ayırır; isimler namespace içinde benzersizdir^[3]
- İlk namespace'ler: `default`, `kube-node-lease`, `kube-public`, `kube-system` ^[3]
- **Namespaced kaynaklar** bir namespace içinde yaşar (Pod, Service, Deployment); **cluster-scoped kaynaklar** namespace'e bağlı değildir (Node, PersistentVolume, StorageClass) ^[3]
- Namespace'ler iç içe olamaz; her kaynak yalnızca **tek namespace'te** bulunur^[3]
- Benzer kaynakları ayırmak için namespace değil **label** kullanılır; `kube-` ön ekli namespace oluşturulmaz^[3]

```
kubectl create namespace lab
kubectl get pods -n lab
kubectl api-resources --namespaced=true
kubectl config set-context --current --namespace=lab
```



Label ve Annotation

- **Label:** tanımlayıcı, **seçilebilir** nitelikler; organizasyon ve seçim için kullanılır^[5]
- **Annotation:** tanımlayıcı olmayan keyfi metadata; **seçim için kullanılmaz**^[6]
- Label key: isteğe bağlı ön ek (DNS subdomain, en fazla 253 karakter) + `/` + isim (en fazla 63 karakter)^[5]
- `kubernetes.io/` ve `k8s.io/` ön ekleri Kubernetes'e ayrılmıştır^[5]
- Bir nesnedeki **tüm annotation'ların** toplam boyutu 256 KiB ile sınırlıdır^[6]



Label Selector

- Selector, nesneleri seçmenin temel aracıdır; label'lar benzersizlik sağlamaz ^[5]
- **Equality:** `=`, `==`, `!=`; virgül = **AND**; OR operatörü yok ^[5]
- **Set-based:** `in`, `notin`, `exists`, `!key` ^[5]

```
kubectl get pods -l environment=production  
kubectl get pods -l 'environment in (production,qa)'  
kubectl get pods -l 'tier in (frontend),environment!=qa'
```

- Controller'lar (Deployment, ReplicaSet, Service) selector'ları kullanır
→ **Workload'lar ve Controller'lar**



Field Selector

- Field selector, **nesne alanlarının değerine** göre seçer; label'a değil^[7]
- Tüm türler `metadata.name` ve `metadata.namespace` alanlarını destekler; diğer alanlar türe göre değişir^[7]
- Yalnızca `=`, `==`, `!=` operatörleri desteklenir; set-based yok^[7]

```
kubectl get pods --field-selector status.phase=Running  
kubectl get pods --field-selector metadata.namespace!=default
```

- **Label selector** etiketlere, **field selector** nesnenin kendi alanlarına göre seçer



Önerilen Ortak Label'lar

- `app.kubernetes.io/*` önerilen ortak label'lar; araçların **birlikte çalışmasını** sağlar^[9]
- Kubernetes "uygulama"yı resmi olarak tanımlamaz; uygulama metadata ile anlatılır^[9]
- Bu label'lar zorunlu değildir; yine de her kaynağa uygulanması önerilir^[9]

Label	Açıklama
<code>app.kubernetes.io/name</code>	Uygulamanın adı (ör. <code>mysql</code>)
<code>app.kubernetes.io/instance</code>	Uygulama örneğini benzersiz tanımlar
<code>app.kubernetes.io/version</code>	Uygulama sürümü (SemVer veya hash)
<code>app.kubernetes.io/component</code>	Mimari içindeki bileşen (ör. <code>database</code>)
<code>app.kubernetes.io/part-of</code>	Üst düzey uygulama adı (ör. <code>wordpress</code>)
<code>app.kubernetes.io/managed-by</code>	Uygulamayı yöneten araç (ör. <code>Helm</code>)



Well-Known Labels, Annotations ve Taints

- `kubernetes.io` ve `k8s.io` ön ekleri Kubernetes'e **ayrılmıştır**; yerleşik label, annotation ve taint adlarının resmi listesi well-known referansındadır^[15]
- Yerleşik adlar ön ek bazında gruplanır^[15]

Örnek	Tür	Kullanım
<code>kubernetes.io/hostname</code>	Label	Node hostname'i
<code>kubernetes.io/arch</code> / <code>kubernetes.io/os</code>	Label	Node mimari/işletim sistemi
<code>node.kubernetes.io/not-ready</code>	Taint	Node sağlıklı → Scheduling ve Yerleşim
<code>topology.kubernetes.io/zone</code>	Label	Bölge (failure domain)

- Kendi label'larında **ön ek** kullanmak (ör. `example.com/uygulama`), yerleşik ve otomasyon label'larıyla çakışmayı önler^[5]



OwnerReferences ve Finalizer

- `metadata.ownerReferences` : bu nesnenin **sahibini** gösterir; `apiVersion`, `kind`, `name`, `uid` ve `blockOwnerDeletion` alanlarından oluşur^[10]
- Controller'lar bunu otomatik set eder (ör. ReplicaSet sahip olduğu Pod'ları işaretler)^[10]
→ `Workload'lar ve Controller'lar`
- `metadata.finalizers` : silmeyi engelleyen liste; liste boşalana dek nesne silinmez (`deletionTimestamp` eklenir)^[11]
- Özel finalizer adları **alan adı (domain) içermelidir**: `example.com/finalizer`; alan adı içermeyen adlar reddedilir^[11]

```
metadata:
  ownerReferences:
  - apiVersion: apps/v1
    kind: ReplicaSet
    name: my-rs
    uid: abc123
  finalizers:
  - example.com/finalizer
```



Özet

- **Manifest**, nesnenin istenen durumunu anlatır: `apiVersion`, `kind`, `metadata`, `spec`
- **Yönetim**: gerçek ortamda declarative (`kubectl apply`); SSA bunun güncel (modern) biçimidir
- **Yapı taşları**: namespace, label, annotation ve selector'lar nesneleri gruplar ve seçer
- **spec vs status** : istenen ile gerçek durum arasındaki fark, control plane'in işidir
- Sonraki adım: Pod ve Pod yaşam döngüsü
→ **Pod ve Pod Yaşam Döngüsü**

Kaynaklar





Kaynaklar

1. Kubernetes API: kubernetes.io/docs/concepts/overview/kubernetes-api/
2. Kubernetes Nesneleri: kubernetes.io/docs/concepts/overview/working-with-objects/
3. Namespace'ler: kubernetes.io/docs/concepts/overview/working-with-objects/namespaces/
4. Nesne İsimleri ve ID'ler: kubernetes.io/docs/concepts/overview/working-with-objects/names/
5. Label ve Selector: kubernetes.io/docs/concepts/overview/working-with-objects/labels/
6. Annotation: kubernetes.io/docs/concepts/overview/working-with-objects/annotations/
7. Field Selector: kubernetes.io/docs/concepts/overview/working-with-objects/field-selectors/
8. Nesne Yönetimi: kubernetes.io/docs/concepts/overview/working-with-objects/object-management/
9. Önerilen Ortak Label'lar: kubernetes.io/docs/concepts/overview/working-with-objects/common-labels/
10. Owner ve Dependent: kubernetes.io/docs/concepts/overview/working-with-objects/owners-dependents/
11. Finalizer: kubernetes.io/docs/concepts/overview/working-with-objects/finalizers/
12. kubectl: kubernetes.io/docs/concepts/overview/kubectl/
13. kubectl Hızlı Referans: kubernetes.io/docs/reference/kubectl/quick-reference/
14. Server-Side Apply: kubernetes.io/docs/reference/using-api/server-side-apply/
15. Well-Known Label/Annotation/Taint: kubernetes.io/docs/reference/labels-annotations-taints/